UNIVERSIDAD TECNOLOGICA NACIONAL

Trabajo Práctico Integrador
Sistema de Gestión Hospitalaria – Anexo IA

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.
Organización Empresarial

Materia
Bases de Datos I

Docente
Carcaño, Nicolás

Alumno
Emperador, Javier

07 de noviembre de 2026

Introducción
El presente anexo documenta los intercambios realizados con herramientas de Inteligencia Artificial como apoyo pedagógico durante el desarrollo del Trabajo Final Integrador de la materia Bases de Datos I.

La herramienta utilizada fue **Claude** (**Anthropic**), empleada con el rol de tutor: guiar el razonamiento, señalar errores, plantear alternativas y acompañar decisiones técnicas, sin reemplazar el trabajo crítico del estudiante. En ningún caso se solicitaron soluciones completas ya terminadas; en cambio, se trabajó de forma iterativa mediante preguntas concretas surgidas durante el proceso de desarrollo.

Los intercambios están organizados por etapa del TFI, en orden cronológico dentro de cada una. Cada intercambio incluye el motivo de la consulta, la orientación recibida y la decisión final tomada. En total se documentan 23 intercambios distribuidos en las cinco etapas del trabajo:

Etapa	Intercambios	Período
Etapa 1 — Modelado y Constraints	6 (1 al 6)	26 de octubre de 2026
Etapa 2 — Carga masiva con SQL puro	5 (7 al 11)	26 de octubre de 2026
Etapa 3 — Consultas avanzadas y reportes	4 (12 al 15)	27 de octubre de 2026
Etapa 4 — Seguridad e integridad	4 (16 al 19)	27 de octubre de 2026
Etapa 5 — Concurrencia y transacciones	4 (20 al 23)	28 de octubre de 2026

Etapa 1: Modelado y Constraints

Estrategia de modelado de enums

```
Estudiante: Los enums del sistema Java (TipoSangre, EspecialidadMedica, EstadoCita)
– ¿los modelamos como columna con CHECK en la tabla o como tabla de referencia
separada con FK?

Claude: Hay dos opciones:

  Opción A: Columna con CHECK en la tabla
  (ej. CHECK (tipo_sangre IN ('A+', 'A-', ...))). Más simple y suficiente para este TPI.

  Opción B: Tabla de referencia separada
  (ej. tabla tipos_sangre con FK). Más ordenado, extensible como sistema y genera
  un modelo más rico.

  La opción A es más simple. La B es más "enterprise" pero agrega más tablas al DER
  y enriquece las consultas de etapas posteriores.

Estudiante: Me gusta la B por cuestión de escalado de sistema.
```

Decisión tomada: Se eligió la Opción B (tablas de referencia separadas) porque permite un diseño más ordenado, con posibilidades de crecimiento, y enriquece el modelo relacional con más entidades.

Las tablas de catálogo resultantes son:

- **tipos_sangre** - valores: A+, A-, B+, B-, AB+, AB-, O+, O-
- **especialidades_medicas** - valores: Cardiología, Neurología, Pediatría, Traumatología, Ginecología, Urología, Oftalmología, Dermatología, Psiquiatría, Medicina General, Cirugía General, Anestesiología
- **estados_cita** - valores: PROGRAMADA, CONFIRMADA, CANCELADA, COMPLETADA

Modelado de herencia y subtablas

```
Claude: En Java Paciente y Medico heredan de Persona (clase abstracta). En una base de
datos relacional no existe herencia, hay dos formas de modelarlo:

  Opción A: Tabla por clase concreta — los campos comunes (nombre, apellido, dni, etc.)
  se repiten en pacientes y medicos. Simple pero con redundancia.

  Opción B: Tabla base personas con FK en pacientes y medicos — sin repetición,
  escalable si se agregan más tipos de personas (enfermeros, administrativos, etc.).

  Para las listas de HistoriaClinica (diagnósticos, tratamientos, alergias) propuse
  una tabla unificada con discriminador (tipo = 'DIAGNOSTICO'/'TRATAMIENTO'/'ALERGIA').

Estudiante: Me gusta la Opción B para la herencia por cuestión de escalado.
Para la historia clínica prefiero tablas separadas por tipo, ya que cada una puede
tener columnas específicas en el futuro (ej. severidad en alergias).
```

Decisión tomada:

- **Herencia → Opción B**: se crea tabla **personas** como base, y **pacientes** y **medicos** tienen un campo **persona_id** (FK a personas). Criterio: escalabilidad del sistema.
- **Subtablas de historia clínica → tablas separadas**: **hc_diagnosticos**, **hc_tratamientos** y **hc_alergias** son tablas independientes con **historia_clinica_id** (FK a historias_clinicas). Criterio: cada tipo puede tener columnas específicas en el futuro sin afectar a los demás.

Etapa 2: Carga masiva con SQL puro

Sintaxis de WITH RECURSIVE + INSERT en MySQL Workbench

```
Estudiante: Al ejecutar el script de carga masiva, MySQL Workbench lanzó este error:
Error Code: 1064. You have an error in your SQL syntax; check the manual that
corresponds to your MySQL server version for the right syntax to use near
'INSERT INTO personas ...' at line 6

El código era:
  WITH RECURSIVE serie AS (...)
  INSERT INTO personas (...) SELECT ... FROM serie;

Claude: MySQL Workbench no acepta WITH RECURSIVE seguido directamente de INSERT INTO.
La solución es materializar la serie en una tabla temporal primero:
  CREATE TEMPORARY TABLE serie (n INT);
  INSERT INTO serie (n) WITH RECURSIVE cte AS (...) SELECT n FROM cte;
  INSERT INTO personas (...) SELECT ... FROM serie;
  DROP TEMPORARY TABLE IF EXISTS serie;
```

Decisión tomada: se usa una tabla temporal **serie** para materializar la secuencia generada por **WITH RECURSIVE** antes de hacer el **INSERT** principal. El **DROP TEMPORARY TABLE IF EXISTS** previo al **CREATE** evita errores si la tabla ya existe de una ejecución anterior interrumpida.

FK constraint con MOD sobre COUNT asumiendo IDs contiguos

```
Estudiante: Al insertar médicos aparece:
Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails
(`hospital_db`.`medicos`, CONSTRAINT `fk_medicos_departamento` FOREIGN KEY
(`departamento_id`) REFERENCES `departamentos` (`departamento_id`))

El código usaba:
  MOD(p.persona_id, (SELECT COUNT(*) FROM departamentos)) + 1

Claude: El problema es que MOD(id, COUNT(*)) + 1 asume que los IDs de departamentos
son contiguos desde 1 hasta N. Si hay cualquier gap (por ejemplo, el primer departamento
fue insertado en Etapa 1 con id=1 y los siguientes en Etapa 2), el resultado puede
apuntar a un ID inexistente. La solución es usar un JOIN con ROW_NUMBER():

  JOIN (
    SELECT departamento_id,
           ROW_NUMBER() OVER (ORDER BY departamento_id) - 1 AS rn,
           COUNT(*) OVER () AS total
    FROM departamentos
  ) d ON d.rn = MOD(sub.persona_id, d.total)

Así siempre se elige un ID real, independientemente de los gaps.
```

Decisión tomada: se reemplazó el patrón **MOD + COUNT + 1** por un JOIN con **ROW_NUMBER() OVER (ORDER BY …)** en todas las tablas donde se asigna una FK mediante distribución modular: **departamentos**, **hospitales**, **medicos**, **salas**.

Etapa 3: Consultas avanzadas y reportes

Diseño del rol del administrador y alcance de las consultas JOIN

```
Claude: Pensando en el sistema hospitalario, ¿qué información necesitaría ver
un administrativo en su pantalla principal? ¿Le interesa más ver el detalle de
una cita completa (quién, con quién, cuándo, en qué sala), o preferiría ver el
plantel de médicos organizado por hospital/departamento?

Estudiante: El administrador se suele abstraer más, así que debe mirar más a
gran escala. Debería mirar por hospital, luego por departamento e ir de mucha
información a información más filtrada. No sé si el administrador es solo uno
y administra todos los hospitales o hay uno por hospital.

Claude: En sistemas hospitalarios reales suele haber un administrador general
(ve todos los hospitales) y administradores por hospital. Para la consulta
conviene diseñarla para el administrador general, así usamos más tablas en el
JOIN y es más representativa. La jerarquía del sistema es:
hospital → departamento → sala → cita.

Propuesta aceptada:
- Q1: resumen de citas por hospital y departamento (cantidades, estados,
  facturación).
- Q2: detalle del plantel médico (médico, especialidad, departamento, hospital).
```

Decisión tomada: Q1 y Q2 siguen la jerarquía del sistema de mayor a menor granularidad. Q1 es el reporte de actividad general; Q2 es el reporte del plantel médico.

Criterio del umbral mínimo para GROUP BY + HAVING

```
Claude: Para Q3 (GROUP BY + HAVING), ¿qué umbral mínimo de citas te parece
razonable para filtrar especialidades con poca actividad?

Estudiante: Me parecen bien 5 citas como mínimo. Así de paso colocamos un
límite para no sobre analizar. Para los casos con menos de 5 citas quizás
corresponde otro criterio, o son situaciones demasiado límite y casos muy
particulares.

Claude: Perfecto. HAVING COUNT(*) > 5 deja afuera los meses y especialidades
con actividad marginal que podrían distorsionar un análisis estadístico.
Los casos con menos de 5 citas merecen otro tipo de seguimiento (por ejemplo,
revisión manual o alerta específica), no un reporte agregado.
```

Decisión tomada: **HAVING COUNT(c.cita_id) > 5** como umbral mínimo en Q3. El reporte mensual filtra especialidades con actividad marginal.

Etapa 4: Seguridad e integridad

Elección del perfil de usuario con privilegios mínimos

```
Claude: Para el usuario con privilegios mínimos, ¿qué rol tiene sentido en el
sistema? Por ejemplo: un usuario recepcion que solo puede consultar pacientes y
citas (solo SELECT), o un usuario medico_app que puede ver y registrar citas
pero no tocar tablas administrativas.

Estudiante: El usuario de recepcion es lo que elijo, principalmente porque es
la opción más sencilla y directa. Comprendo que buscamos usuarios con
privilegios mínimos.

Claude: Exacto. Mínimos privilegios significa darle al usuario solo lo que
necesita para su trabajo y nada más. recepcion con solo SELECT sobre las
tablas que necesita ver es perfecto para demostrarlo. Sin acceso a
historias_clinicas, hc_diagnosticos, hc_tratamientos ni hc_alergias, que son
datos médicos sensibles.
```

Decisión tomada: usuario **recepcion@localhost** con SELECT únicamente sobre tablas operativas (pacientes, personas, citas, medicos, especialidades, estados_cita, hospitales, departamentos, salas). Sin acceso a historias clínicas ni tablas de catálogo médico sensible.

Etapa 5: Concurrencia y transacciones

Por qué se necesitan dos pestañas para simular un deadlock

```
Estudiante: ¿Hace falta dos pestañas separadas o puedo ejecutar todo desde
la misma pestaña de query que tengo abierta?

Claude: La necesidad es que un deadlock requiere dos transacciones abiertas
al mismo tiempo, en paralelo. Si ejecutás todo desde una sola pestaña, los
comandos se ejecutan en secuencia: primero termina uno, después arranca el otro.
Nunca van a estar los dos bloqueados al mismo tiempo, así que el deadlock
nunca ocurre.

Con dos pestañas podés dejar la Sesión A con su transacción abierta mientras
ejecutás los comandos de Sesión B. Cuando B intenta bloquear una fila que A
ya tiene, queda esperando. Recién ahí, cuando volvés a A y ésta intenta
bloquear la fila que B tiene, MySQL detecta el ciclo y emite Error 1213.
```

Decisión tomada: se usaron dos pestañas de query en MySQL Workbench, cada una con su propia conexión independiente al servidor, para ejecutar las transacciones en paralelo.

Variables de sesión no compartidas entre conexiones

```
Estudiante: Ejecuté el SET @cita_a = 1 en una pestaña pero el UPDATE
sigue devolviendo Rows matched: 0.

Claude: Las variables de sesión (@nombre) son locales a cada conexión.
Si seteás @cita_a en la Pestaña A, la Pestaña B no la ve — cada pestaña
de Workbench abre una conexión independiente al servidor. Tenés que ejecutar
el SET en cada pestaña por separado antes de arrancar.

Estudiante: Ahora funciona, los UPDATE matchean correctamente.
```

Decisión tomada: el bloque **SET @cita_a** y **SET @cita_b** se ejecutó en cada sesión por separado antes de iniciar las transacciones.